Guardian Agent: A Bayesian Personalized Weather Hazard Safety Agent

Erfan Zarafshan [1] **Maby Gavilan Abanto** [1]

Abstract

Existing weather emergency alert systems are reactive and broadcast-style: the same warning text is sent to every recipient in an affected region, regardless of their personal vulnerability. We present *Guardian Agent*, a context-aware AI system that combines real-time weather data, a gradient-boosted hazard classifier trained on the NOAA Storm Events Database, and a discrete Bayesian network with ten nodes to produce a personalized risk posterior, which a deterministic rule planner then maps to an ordered list of recommended actions. The system was implemented in Python, deployed as an interactive web demonstration, and evaluated on hold-out data and four hazard scenarios. The trained classifier reaches ROC-AUC $= 0.759$ on 21,281 held-out county-day cells, and the end-to-end system produces visibly distinct recommendations for users with different vulnerability profiles facing identical hazards, demonstrating the value of personalized probabilistic reasoning over generic broadcast warnings.

1. Introduction

Severe weather events kill hundreds and cause billions of dollars in property damage in the United States each year. The dominant public-warning mechanism, Wireless Emergency Alerts (WEA), pushes the same short message to every cell phone in a geographic polygon (Federal Communications Commission, 2024). Two recipients in the same neighborhood: one living on the third floor of a sturdy building with a high-clearance vehicle; the other on the ground floor with a mobility limitation and no car, receive identical instructions, even though the appropriate response should be different. The result is a well-documented problem of *alert fatigue* (Rush et al., 2016; Sutton & Wood, 2025; Gani et al., 2025; Newton et al., 2026) in which recipients learn to discount warnings that do not appear to apply to them.

This paper addresses the mentioned gap by treating this generic recommendation problems as a probabilistic inference task over evidence based on the user needs. Our system, **Guardian Agent**, gets a sense of the current weather events through public weather APIs, uses those observations along with a learned threat signal and user profile, and computes a posterior over four risk levels {Low, Moderate, High, Critical} for that specific user. A rule planner then converts the argmax of the posterior into an ordered list of actions chosen from a fixed catalog (notify, shelter, evacuate, activate flood lights, etc.), each annotated with a human-readable rationale.

Contributions. (1) An agent architecture that integrates a machine learning classifier and a Bayesian network into a single perceive-reason-act loop. (2) A discrete Bayesian network whose conditional probability tables are specified by scoring functions rather than enumerated by hand, scaling to ten nodes while remaining auditable. (3) A publicly deployed interactive demonstration at `https://guardianagent.streamlit.app/` that lets visitors explore both real and hypothetical hazards under multiple personal profiles.

2. Methodology

2.1. Theoretical background

2.1.1. Gradient Boosting Machines

The Gradient Bossting Machines (Friedman, 2001) are a machine learning algorithm that adds new models, known as learners, in a sequential fashion. The procedure of learning consists on providing the resulting gradients from the previous learners to perform the optimization of an arbitrary (but differentiable) loss function.

Consider $(\mathbf{x}_i, y_i)_{i=1}^N$ be the training set. The goal is to find a function $f(\mathbf{x})$ that minimises an expected loss $\mathbb{E}[L(y, F(\mathbf{x}))]$. GBMs approximate this minimisation in function space by iterative gradient descent:

[1]Division of Computer Science and Engineering, Louisiana State University, Baton Rouge, LA, USA. Correspondence to: Erfan Zarafshan <ezaraf1@lsu.edu>.

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \rho\, h_m(\mathbf{x}, \theta), \quad m = 1, \ldots, M \quad (1)$$

where $\rho \in (0, 1]$ is the *learning rate* (or shrinkage), and h_m is a base learner fitted to the *pseudo-residuals*

$$r_{im} = -\left[\frac{\partial L(y_i,\, f(\mathbf{x}_i))}{\partial f(\mathbf{x}_i)}\right]_{f=f_{m-1}}. \quad (2)$$

The pseudo-residuals represent the negative gradient of the loss with respect to the current prediction; fitting a tree to them is therefore a functional gradient-descent step.

Regularisation is achieved through the learning rate ρ, the maximum tree depth, and sub-sampling of observations and features (Natekin & Knoll, 2013). The pseudo-algorithm is described below:

Algorithm 1: Friedman's Gradient Boost Algorithm

Inputs:
- input data $(x, y)_{i=1}^{N}$
- number of iterations M
- arbitrary function loss-function $\Psi(y, f)$
- choice of base learner model $h(x, \theta)$

Algorithm:
1. initialize $\widehat{f}_0$ with a constant.
2. **for** $t = 1$ to M **do**:
3. compute the negative gradient $g_t(x)$
4. fit a new base-learner function $h(x, \theta_t)$
5. find the best gradient descent step-size ρ_t:
   $\rho_t = \arg\min_\rho \sum_{i=1}^{N} \Psi\left[y_i, \widehat{f}_{t-1}(x_i) + \rho h(x_i, \theta_t)\right]$
6. update the function estimate:
   $\widehat{f}_t \leftarrow \widehat{f}_{t-1} + \rho_t h(x, \theta_t)$
7. **end for**

2.1.2. Bayesian Networks

A Bayesian Network (BN) is a probabilistic graphical model that encodes the joint distribution over a set of random variables through a directed acyclic graph (DAG) and a collection of conditional probability tables (CPTs) (Russell & Norvig, 2020).

Structure

Let $\mathbf{X} = \{X_1, \ldots, X_d\}$ be a set of discrete random variables. A BN is a pair $\mathcal{B} = (\mathcal{G}, \mathbf{\Theta})$ where $\mathcal{G} = (\mathbf{X}, \mathcal{E})$ is a DAG whose nodes correspond to variables and whose edges encode direct dependencies, and $\mathbf{\Theta}$ is the set of CPTs. The DAG encodes the *Markov condition*: every variable is conditionally independent of its non-descendants given its parents $\mathrm{Pa}(X_i)$.

Joint distribution

Applying the chain rule and using the Markov condition yields the fundamental factorisation

$$P(X_1, \ldots, X_d) = \prod_{i=1}^{d} P(X_i \mid \mathrm{Pa}(X_i)). \quad (3)$$

For a discrete BN (as implemented in `pgmpy`'s `DiscreteBayesianNetwork`), each factor $P(X_i \mid \mathrm{Pa}(X_i))$ is stored as a CPT whose rows index the parent configurations and whose columns index the states of X_i.

Inference

Given evidence $\mathbf{e}$ on a subset of variables, *posterior inference* computes $P(X_q \mid \mathbf{e})$ for a query variable X_q. Exact methods such as variable elimination and the junction-tree algorithm exploit the factorisation to avoid enumerating the full joint (Koller & Friedman, 2009); approximate methods, such as MCMC, are used when the graph is too dense for exact treatment.

2.2. Architecture overview

In figure 1, the Guardian Agent's perceive–reason–act cycle. Each component is a Python module under `src/guardian/`; modules are loosely coupled through typed `pydantic` dataclasses so that each layer can be tested and swapped independently.

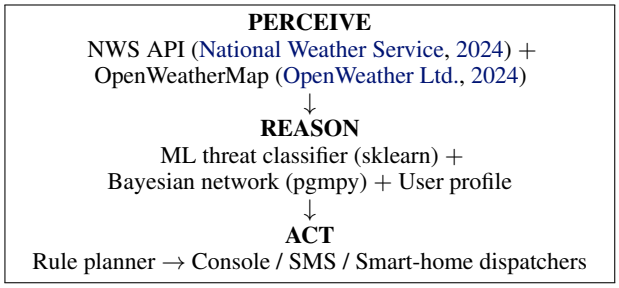

Figure 1. Architecture. Each layer is a separate Python module communicating through typed dataclasses.

2.3. Perception layer

The perception layer queries two additional sources. The U.S. National Weather Service API (National Weather Service, 2024) provides hazard *alerts* (event name, severity, urgency, certainty) indexed by zone code; OpenWeatherMap (OpenWeather Ltd., 2024) provides current *conditions* (temperature, wind speed, precipitation rate, humidity). An aggregator merges both into a single `WeatherObservation` record. If either source fails, the system falls back to the other and logs a `partial observation` flag rather than crashing.

2.4. Threat classifier

The classifier is trained to predict, given a (county, day) cell, the probability that any severe weather event will occur in the following 24 hours. Training data is the NOAA Storm Events Database (National Oceanic and Atmospheric Administration, 2024): we used 2018–2024 events from five Gulf Coast states (LA, TX, MS, AL, FL), yielding $106{,}401$ event records. We synthesized one positive cell per event and sampled negative cells uniformly from the same county-time distribution, producing $106{,}401$ training rows after class-balancing weights are applied.

Features are intentionally shallow: month and day-of-year (cyclically encoded as $\sin / \cos$), county FIPS code (categorical), state indicator, and a 30-day rolling event count for the same county. We deliberately avoid radar or atmospheric soundings to keep the prototype reproducible from a free public dataset; a production system would naturally consume those.

The model is a Gradient Boosting Classifier (Friedman, 2001) from `scikit-learn` (Pedregosa et al., 2011). Hyperparameters were selected by cross-validation (5 folds) over a small grid (`n_estimators` $\in \{50, 100, 200\}$, `max_depth` $\in \{3, 5\}$, `learning_rate` $\in \{0.05, 0.1\}$). Class imbalance ($\sim 1\%$ positive rate) is handled by inverse-frequency `sample_weight`.

2.5. Bayesian risk engine

The reasoning core is a discrete Bayesian network with 10 nodes arranged as a polytree, implemented in `pgmpy` (Ankan & Textor, 2024; Ankan & Panda, 2015). The network combines three classes of evidence:

- **Hazard evidence** from NWS alerts: `HazardSeverity` (5 states), `Urgency` (5 states).
- **Environmental evidence** from OpenWeatherMap: `WindCategory`, `PrecipCategory` (4 states each).
- **Threat-prediction evidence** from the ML classifier: `EmergingThreat` (3 states, derived by binning the classifier probability).
- **Personal evidence** from the user profile: `HomeFloor`, `VehicleClearance`, `MobilityLimited`.

These eight nodes feed a latent `HazardImpact` node, which in turn determines the query node `RiskLevel` $\in$ {Low, Moderate, High, Critical}. Inference is exact via Variable Elimination (Koller & Friedman 2009, Ch. 9), and completes in a few milliseconds per query at this network size.

A practical issue with hand-built networks is the combinatorial blowup of CPT entries (the parents of `HazardImpact` alone produce $5{\times}5{\times}4{\times}4{\times}3 = 1200$ rows). To avoid hand-enumeration, we specify each CPT through a small scoring function that maps parent assignments to a numeric impact score, then bucketizes that score into the child's discrete states. This collapses CPT authorship into roughly fifteen lines of readable Python per node and produces tables that are internally consistent across parent combinations.

2.6. Action planner

The planner is intentionally deterministic. It takes the argmax of the risk posterior, the user profile, and the active alert and selects actions from a catalog of 8 elements: {NOTIFY_USER, NOTIFY_CONTACTS, RECOMMEND_SHELTER, RECOMMEND_EVACUATE, ACTIVATE_FLOOD_LIGHTS, SOUND_ALARM, ADJUST_THERMOSTAT, LOG_ONLY} across three dispatch channels (console, SMS, smart-home). Message templates related to hazard are applied per alert event (e.g., a flash flood template warns about flooded roadways; a tornado template specifies interior sheltering). User-preference fields gate non essential channels: quiet-hours suppress *moderate* level notifications but are overridden for *critical* life-safety actions.

Every action carries a `rationale` string that is written to the log alongside the action. This converts the system's behavior into a post-hoc auditable transcript — in our judgment a more important property than maximizing recommendation accuracy on a benchmark, given the consequences of incorrect emergency advice.

3. Experimental Results

3.1. Operational environment

The system was developed in Python 3.13 on macOS 14.5 (Apple Silicon). Key libraries: `pgmpy` 1.1, `scikit-learn` 1.6, `pandas` 2.2, `pydantic` 2.10, `streamlit` 1.32, `plotly` 5.18. SMS dispatch is integrated via Twilio in dry-run mode by default; the deployed web demo enforces dry-run unconditionally to prevent abuse. External data: NOAA Storm Events 2018–2024 (Gulf Coast states), NWS-API (live), OpenWeatherMap free-tier (live). Source code at `https://github.com/ErfanZarafshan/guardian-agent`.

3.2. Classifier evaluation

We evaluate the trained Gradient Boosting Classifier on a stratified $80/20$ split of the prepared cells.

ROC-AUC of 0.759 is well above chance (0.500) but well below the 0.85–0.95 that mature meteorological nowcasting

Table 1. Threat classifier performance on the held-out test split.

Metric	Value
Training cells	85,120
Test cells	21,281
Positive base rate	$\approx 0.9\%$
Test ROC-AUC	**0.759**
Test accuracy	0.696
Test F_1	0.041

systems achieve with radar input. We view this as the honest ceiling of a calendar-and-location-only feature set; the Bayesian fusion downstream of the classifier compensates partially by leaning more heavily on NWS alert evidence when the classifier signal is weak.

The very low F_1 reflects the rare-event nature of the task: the classifier rarely predicts a positive at the default 0.5 threshold. Operationally we do not threshold; we bin the classifier probability into three buckets (Low, Moderate, High) and feed those as discrete `EmergingThreat` evidence into the Bayesian network, which is what actually drives downstream behavior.

3.3. End-to-end personalization

The crucial test of personalization is whether the same hazard produces different recommendations for different users. Table 2 reports posteriors for two reference profiles — *Alex* (third-floor apartment, owns vehicle, no medical limitations) and *Sam* (ground-floor apartment in flood zone AE, no vehicle, mobility-limited) — across four scenarios.

Table 2. Posterior probability of each `RiskLevel` state by scenario, for two contrasting user profiles. Argmax in **bold**. "Argmax flip" indicates that the planner selects a strictly different action set for the two users.

	Low	Mod.	High	Crit.
Clear day				
Alex	**.81**	.13	.05	.01
Sam	**.78**	.12	.07	.03
Flash Flood Watch (argmax flip)				
Alex	.26	**.37**	.27	.11
Sam	.13	.22	**.38**	.28
Tornado Warning				
Alex	.02	.12	.37	**.48**
Sam	.00	.07	.32	**.61**
Tropical Storm Warning (argmax flip)				
Alex	.06	.19	**.39**	.37
Sam	.02	.09	.34	**.56**

In two of four scenarios (Flash Flood Watch and Tropical Storm Warning) the posterior argmax differs between Alex and Sam. Because the planner conditions on the argmax, this difference falls into a notorious different action set: Sam receives a shelter recommendation where Alex receives only a notification, and Sam's higher-priority contact list is paged where Alex's is not. In the Tornado scenario both users are placed at Critical, but Sam's posterior assigns ~ 13 percentage points more mass to Critical, reflecting the amplifying effect of mobility limitation on tornado outcomes.

3.4. Live agent and dispatch behaviour

Running the agent against live NWS and OpenWeatherMap data for our profile address in Baton Rouge, LA on a calm afternoon produced a posterior of $\{$Low: 0.485, Moderate: 0.319, High: 0.138, Critical: $0.057\}$. A subsequent three-cycle continuous run with a 60-second polling interval verified the deduplication logic: the fingerprint (active alert events $\times$ argmax) matched between cycles 2 and 3, suppressing $N > 0$ non-console actions per duplicate cycle while keeping the console alive, avoiding that an hour-long flash flood warning would generate twelve identical SMS messages to each contact.

3.5. Test coverage and code quality

The code base ships with 239 unit and integration tests in 9 test files exercising every layer (profile validation 27 tests, weather parsers 41, classifier 33, Bayesian engine 21, risk engine integration 14, planner 37, dispatchers 19, agent loop 15, plus smoke tests). All pass on Python 3.13, including end-to-end tests that mock the weather aggregator and exercise the full perceive–reason–act loop.

4. Conclusions and Future Work

Guardian Agent shows that combining probabilistic reasoning with ML and modern web tooling produces a system that differentiates its safety guidance based on user-specific factors, gap that current broadcast emergency alerts do not address. The implemented Bayesian network is small enough to be auditable and large enough to capture the interaction between hazard severity, ML threat prediction, and personal vulnerability; the rule planner is deterministic and explicable, with every action carrying a written rationale.

The most valuable extensions, in priority order, would be (1) replacing the calendar-and-location-only classifier features with NEXRAD radar reflectivity nowcasts to lift ROC-AUC into the $0.85+$ band; (2) shipping the smart-home dispatcher as a real Alexa Skills Kit integration so the existing `SmartHomeDispatcher` interface can be applied end-to-end on a real device; (3) replacing the unconditional dry-run constraint on the deployed web demo with per-user verified-recipient SMS, allowing users to receive real notifications while still preventing abuse; and (4) learning the Bayesian network's CPT scoring functions from historical NWS alert + outcome data instead of specifying them by

hand. None of these change the architecture, they replace specific components behind unchanged interfaces.

References

Ankan, A. and Panda, A. pgmpy: Probabilistic graphical models using Python. In *Proceedings of the 14th Python in Science Conference (SciPy)*, pp. 6–11, 2015. doi: 10.25080/Majora-7b98e3ed-001.

Ankan, A. and Textor, J. pgmpy: A Python toolkit for Bayesian networks. *Journal of Machine Learning Research*, 25(265):1–8, 2024. URL http://jmlr.org/papers/v25/23-0487.html.

Federal Communications Commission. Wireless emergency alerts (WEA). https://www.fcc.gov/consumers/guides/wea-capable-mobile-devices, 2024. Accessed: 2026-04-01.

Friedman, J. H. Greedy function approximation: A gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001. doi: 10.1214/aos/1013203451.

Gani, I., Litchfield, I., Shukla, D., Delanerolle, G., Cockburn, N., and Pathmanathan, A. Understanding "alert fatigue" in primary care: Qualitative systematic review of general practitioners attitudes and experiences of clinical alerts, prompts, and reminders. *Journal of Medical Internet Research*, 27:e62763, February 2025. ISSN 1438-8871. doi: 10.2196/62763. URL http://dx.doi.org/10.2196/62763.

Koller, D. and Friedman, N. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, MA, 2009.

Natekin, A. and Knoll, A. Gradient boosting machines, a tutorial. *Frontiers in Neurorobotics*, 7, 2013. ISSN 1662-5218. doi: 10.3389/fnbot.2013.00021. URL http://dx.doi.org/10.3389/fnbot.2013.00021.

National Oceanic and Atmospheric Administration. NOAA storm events database. https://www.ncdc.noaa.gov/stormevents/, 2024. Accessed: 2026-04-01.

National Weather Service. NWS API web service. https://www.weather.gov/documentation/services-web-api, 2024. Accessed: 2026-04-01.

Newton, N., Bamgboje-Ayodele, A., Forsyth, R., Tariq, A., Huang, J., Yannam, R., Lalor, D. J., Sobey, A. J., and Baysari, M. T. Experiences of alert fatigue and its contributing factors in hospitals: Qualitative study. *Journal of Medical Internet Research*, 28:e78676–e78676, February 2026. ISSN 1438-8871. doi: 10.2196/78676. URL http://dx.doi.org/10.2196/78676.

OpenWeather Ltd. OpenWeatherMap API. https://openweathermap.org/api, 2024. Accessed: 2026-04-01.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Rush, J. L., Ibrahim, J., Saul, K., and Brodell, R. T. Improving patient safety by combating alert fatigue. *Journal of Graduate Medical Education*, 8(4):620–621, October 2016. ISSN 1949-8349. doi: 10.4300/JGME-D-16-00186.1. URL https://doi.org/10.4300/JGME-D-16-00186.1.

Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson, Hoboken, NJ, 4 edition, 2020.

Sutton, J. and Wood, M. M. Opting out: Over-alerting and warning fatigue in the era of wireless emergency alerts. *Journal of Contingencies and Crisis Management*, 33(3):e70076, 2025. doi: https://doi.org/10.1111/1468-5973.70076. URL https://onlinelibrary.wiley.com/doi/abs/10.1111/1468-5973.70076.